Chapter 1
Introduction

1.1 Project Overview

Mobility is one of the most essential needs in human life, especially for individuals who face physical challenges, permanent disabilities, old age limitations, or medical recovery conditions. For such users, a wheelchair is not just a device—it is a tool of independence, dignity, and daily functionality. Traditional manual wheelchairs require continuous physical effort, which many users cannot provide. Even most commercial electric wheelchairs are highly expensive, lack modern smart features, and do not always offer the flexibility needed by different types of users. This creates a strong need for a smart, affordable, customizable mobility solution that makes mobility easier, safer, and more accessible.

This project presents a Smart Electric Wheelchair built using modern embedded technologies. The system is based on the ESP32 Devkit V1 microcontroller, chosen for its processing power, built-in Bluetooth, dual-core architecture, sensor compatibility, and multitasking capability. The wheelchair uses BTS7960 high-power motor drivers to control the left and right motors with stable, smooth, and high-torque operation. Together, these components create a robust and efficient motor control system.

A major highlight of the project is the multi-mode control system, giving users the freedom to choose the most comfortable and accessible method for driving the wheelchair. The system supports four fully functional control methods:

1. Mobile App Button Control

- The custom-designed mobile application allows the user to control the wheelchair through onscreen buttons such as Forward, Backward, Left, Right, and Stop. This method is simple, intuitive, and user-friendly.

2. Voice Control (Inside the App)

- The application includes built-in voice recognition. The user can issue commands like “Forward”, “Left”, “Stop”, etc., making the wheelchair extremely accessible for users with limited hand or finger movement. The app converts speech into Bluetooth signals which the ESP32 instantly processes.

3. Hand-Gesture Control (Inside the App)

- The mobile app also features gesture-based interaction. The user can swipe Up, Down, Left, or Right on the screen to move the wheelchair. Gesture control provides a modern, touch-sensitive interface that is easy to use and requires minimal physical effort.

4. Physical Joystick Control

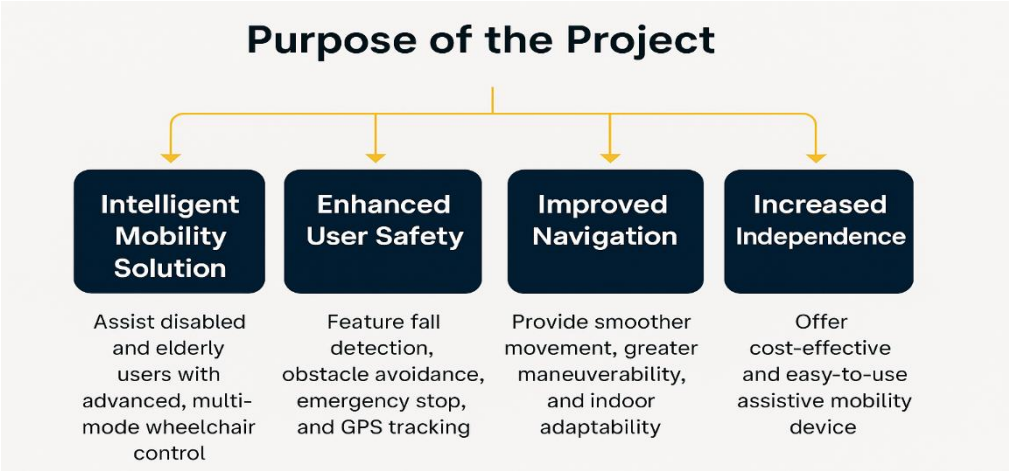
- A physical joystick module is included for manual and traditional operation. This gives the user the flexibility to control the wheelchair without relying on the mobile app, ensuring accessibility in all situations—even when the phone battery is low or unavailable.
- Along with flexible control, the project integrates several advanced safety features. The system uses an MPU6050 accelerometer + gyroscope to implement a real-time fall detection system. If the wheelchair tilts dangerously or experiences sudden impact, the system

immediately stops the motors, activates an alarm, and sends the GPS location for safety. An ultrasonic obstacle detection module continuously scans the path ahead. If an obstacle is detected within 20 cm, the system warns the user with beeps and stops the wheelchair to avoid accidents.

- A physical emergency button is added for highly critical situations. When pressed, it immediately stops all movement, activates an emergency siren pattern, and can send an alert through Bluetooth. This ensures the user always has full control over their safety.
- The project also includes a GPS tracking system using a NEO-6M module. The ESP32 reads the GPS data every few milliseconds and sends the location at regular intervals through Bluetooth. This feature is valuable for caregivers, especially during outdoor travel, allowing monitoring of the wheelchair's position in real time.
- Overall, the Smart Electric Wheelchair integrates motor control systems, real-time sensors, Bluetooth communication, mobile app technologies, safety mechanisms, and intelligent automation into one unified platform. The project aims to provide a smart mobility solution that is affordable, reliable, and highly accessible. It demonstrates how embedded systems can significantly improve the quality of life for people with mobility limitations, offering them more independence, confidence, and safety in daily life.

1.2 Purpose of the Project

- The main purpose of the “ESP32-Based Smart Electric Wheelchair with Multi-Mode Control, Fall Detection, Obstacle Avoidance, and GPS Tracking System” is to create an intelligent, safe, and user-friendly mobility solution for physically disabled and elderly individuals. The system aims to overcome the limitations of traditional wheelchairs by providing multiple control options—mobile app button control, voice commands, gesture control, and joystick operation—ensuring accessibility for users with different physical abilities.
- The project also focuses on enhancing user safety through features such as real-time fall detection, automatic obstacle avoidance, emergency stop functionality, and GPS-based location tracking. These features help prevent accidents, improve response during emergencies, and allow caregivers to monitor the user’s position. By integrating smart sensors, wireless communication, and advanced motor control, the wheelchair provides smoother navigation, higher maneuverability, and better adaptability to indoor environments.
- Overall, the purpose of this project is to deliver a modern assistive mobility device that increases independence, comfort, safety, and convenience for users while remaining cost-effective and easy to operate.



1.3 Key Features

1. Four control methods in one system:
   - App Controller Mode – On-screen buttons for Forward, Back, Left, Right, Stop.
   - Voice Control Mode – App listens to commands like "Forward", "Left", "Stop", etc.
   - Hand-Gesture Mode – Swipe gestures (up/down/left/right) for movement.
   - Physical Joystick – Manual control for users who prefer traditional joystick operation.
2. High-power motor driving: BTS7960 43A drivers provide stable, high-torque movement for heavy wheelchair motors.
3. ESP32 smart controller: Handles Bluetooth, joystick input, GPS, PWM signals, and multitasking smoothly.
4. GPS tracking: Real-time location monitoring for safety and outdoor usage.
5. Smooth speed and turning: PWM-based acceleration ensures controlled forward, backward, and turning movement.
6. User-friendly mobile app: Simple interface to switch between controller, voice, and gesture modes easily.
7. Reliable & safe operation: Overcurrent protection, common ground management, and support for emergency stop.
8. Long battery runtime: Powered by a 12V battery with a stable 5V buck converter for the ESP32.
9. Fully expandable: Future support for obstacle detection, IoT cloud tracking, and autonomous navigation.

1.4 Technology Used

1. ESP32 Devkit V1 Microcontroller
   - Acts as the central processing unit of the system.
   - Provides dual-core processing, built-in Bluetooth Classic, high-speed GPIOs, ADC support, PWM generation, and UART communication.
   - Responsible for motor control, Bluetooth command processing, sensor integration, GPS handling, and safety monitoring.
2. BTS7960 (IBT-2) High-Power Motor Drivers
   - Dual H-bridge motor drivers capable of handling up to 43A current.
   - Used to control the left and right wheelchair motors independently.
   - Supports bidirectional PWM control for smooth acceleration, turning, and braking.
3. Mobile Application (Bluetooth-Based Control)

The mobile app provides three different control modes:

1. Button Controller Mode – Sends directional commands (U, D, L, R, S) to the ESP32 via Bluetooth.
2. Voice Control Mode – Converts speech to control commands inside the app and transmits them to the ESP32.
3. Hand-Gesture Mode – Uses touchscreen gestures (swipe up/down/left/right) to control wheelchair movement.

This multi-interface design makes the wheelchair accessible to users with different ability levels.

4. Physical Joystick Module
    - Analog joystick connected to the ESP32's ADC pins.
    - Provides manual control without needing a mobile phone.
    - Supports differential motor control through X-Y axis mapping and PWM output.
5. MPU6050 Accelerometer and Gyroscope
    - Measures acceleration and tilt angles for fall detection.
    - Enables the system to detect dangerous tilt, overturning, or free-fall conditions.
    - Automatically triggers motor shutdown and emergency alerts when a fall is detected
6. Ultrasonic Sensor (Obstacle Detection)
    - Continuously measures distance to objects in front of the wheelchair.
    - Detects obstacles within a 20 cm safety range.
    - Activates warning tones and can stop the motors to prevent collisions.
7. GPS Module (NEO-6M or Equivalent)
    - Provides real-time geographic coordinates.
    - Sends location data periodically to the app via Bluetooth.
    - Used during emergencies (fall or alarm activation) to share the current location.
8. Emergency Button
    - Physical hardware button used to immediately stop all motor activity.
    - Activates an alarm and sends a GPS alert through Bluetooth.
    - Enhances overall system safety and user confidence.
9. Buzzer and LED Indicators
    - Buzzer generates audible alerts for obstacles, fall alarms, and emergency conditions.
    - The LED indicator displays system activity states (active control, alarm mode, idle mode).
10. Power Management System
    - Powered with a 12V wheelchair battery.
    - Buck converter supplies regulated 5V to the ESP32 and control electronics.
    - Motor drivers draw power directly from the 12V line, ensuring stable high-current performance.
11. Embedded Software (Arduino Framework)
    - Implemented in C/C++ using the Arduino IDE.
    - Manages Bluetooth communication, PWM motor driving, joystick mapping, fall detection logic, ultrasonic processing, GPS parsing (Tiny_GPSPlus), and alarm handling.
    - Use non-blocking programming for smooth multitasking and real-time response.

1.5 Rationale and Motivation

- Mobility assistance is a critical requirement for individuals with physical disabilities, elderly users, and patients recovering from medical conditions. Traditional manual wheelchairs demand continuous physical effort, which many users cannot provide. While electric wheelchairs exist, they are often costly, difficult to customize, and lack modern intelligent features such as voice control, gesture control, fall detection, and emergency alerts. These gaps clearly indicate the need for a smarter, safer, and more accessible mobility solution.
- The primary motivation behind this project is to develop a technologically advanced yet cost-effective smart wheelchair that enhances independence, safety, and ease of use. By integrating embedded systems, wireless control, and real-time sensing, the wheelchair can adapt to different users and environments more effectively than traditional systems.
- To address the diverse physical abilities of different users, the project incorporates multiple control methods—mobile app button control, voice control, hand-gesture control, and physical joystick. These features ensure that the wheelchair can be operated even by individuals with limited hand strength, speech ability, or mobility restrictions. Bluetooth communication provides a reliable and user-friendly way to interface with the wheelchair through a smartphone.
- Safety is another major motivation. Falls, obstacles, and sudden accidents can cause severe injuries. To tackle this, the system integrates an MPU6050-based fall detection mechanism, an ultrasonic obstacle detection unit, and a dedicated emergency button. These features ensure immediate motor shutdown, alarms, and emergency alerts to prevent harm to the user. The addition of GPS tracking further enhances safety by providing real-time location updates in emergencies.
- From an engineering perspective, this project is motivated by the need to demonstrate how modern microcontrollers such as the ESP32, combined with high-current motor drivers like the BTS7960, can create a robust intelligent mobility platform. The project also showcases the practical integration of sensors, wireless communication, and embedded control algorithms into a real-world application.
- Overall, the motivation for this project arises from the desire to improve the quality of life for wheelchair users, provide them with greater independence, and design a system that is safe, responsive, reliable, and technologically advanced—while still being affordable and customizable.

1.6 Goals and Scope

1.6.1 Goals

The primary goals of this project are:

- To design and implement a multi-control intelligent wheelchair that can be operated through a mobile app, voice commands, hand gestures, and a physical joystick.
- To enhance mobility and accessibility for individuals with physical disabilities by providing easy-to-use and flexible control options.

- To ensure user safety through integrated fall detection, obstacle detection, emergency control, and motor shutdown mechanisms.
- To provide real-time location tracking using GPS for monitoring, navigation, and emergency assistance.
- To develop a reliable and responsive embedded control system using the ESP32 microcontroller and BTS7960 high-power motor drivers.
- To deliver a cost-effective solution that can match or surpass the functionality of commercial smart wheelchairs while remaining affordable and customizable.
- To demonstrate efficient integration of hardware, sensors, and software, showcasing practical applications of embedded systems, wireless communication, and control algorithms.

1.6.2 Scope

The scope of this project covers both hardware and software aspects required to create a fully functioning smart wheelchair. The key components included in the project scope are:

Hardware Scope

- Use of ESP32 Devkit V1 as the central controller.
- Motor control using BTS7960 high-power motor drivers.
- Physical joystick for manual analog control.
- Mobile app with button control, voice control, and hand-gesture control via Bluetooth.
- MPU6050 sensor for fall detection.
- Ultrasonic sensor for obstacle detection.
- Emergency stop button for user safety.
- GPS module for real-time location tracking and alerts.
- 12V battery with regulated power management. **Software Scope**
- ESP32 firmware for Bluetooth input processing, voice and gesture command mapping, and joystick handling.
- Fall detection and obstacle avoidance algorithms.
- Emergency alarm activation and siren handling.
- GPS data parsing and location transmission.
- PWM-based motor control with differential steering.
- LED and buzzer indicators for status alerts.
- Timeout mechanisms for inactive Bluetooth or joystick control.

Functional Scope

- Forward, backward, left, right, and stop movement control.
- Automatic switch between Bluetooth and joystick control.
- Real-time safety monitoring and automatic motor shutdown.
- Periodic and emergency GPS location transmission.
- User alerts through buzzer, LED, and Bluetooth messages

CHAPTER 2

Literature Review & Existing Systems

2.1 Literature Survey

- The development of smart wheelchairs has evolved rapidly with advancements in embedded systems, sensor technology, and human–machine interaction. Traditional manual wheelchairs required high physical effort, while early electric wheelchairs relied mainly on joysticks—making them unsuitable for users with limited hand mobility. This led researchers to explore more intelligent and accessible control methods.
- Bluetooth-based wheelchair systems are widely studied because they are low-cost and easy to implement. These typically use smartphones to send basic directional commands to microcontrollers like Arduino or ESP32. However, most studies focus only on simple button controls and lack essential safety features such as collision avoidance or fall detection. Voice-controlled wheelchairs have also been proposed, offering hands-free operation, but they often face accuracy issues in noisy environments and require expensive speech modules.

- Gesture-controlled systems using accelerometers, gyroscopes, or cameras provide an alternative for users who cannot speak or use joysticks. Although promising, these systems are sensitive to environmental factors, require frequent calibration, and are mostly limited to lab prototypes rather than real practical setups.
- Safety mechanisms such as ultrasonic obstacle detection and IMU-based fall detection have been explored extensively in robotics, but very few wheelchair systems combine both methods into a single integrated solution. GPS-based tracking has been used in healthcare for outdoor monitoring, yet most systems depend on GSM or Wi-Fi modules, which increases cost and complexity. Bluetooth-based GPS reporting remains uncommon.
- Many existing designs still rely on low-power motor drivers like L298N and L293D, which are not suitable for real wheelchair motors. Only limited research discusses high-current drivers like BTS7960, which are capable of handling heavy loads and providing smoother PWM control.
- Overall, the literature shows that most research focuses on isolated features—voice control alone, gesture control alone, obstacle detection alone—without offering a complete system that integrates multi-mode control, safety monitoring, emergency handling, and GPS tracking into one practical, real-world wheelchair solution.

2.2 Problem Statement

- Despite the advancements in assistive mobility technologies, many wheelchair users continue to face challenges in daily movement due to limitations in existing systems. Manual wheelchairs require continuous physical effort, making them unsuitable for users with severe disabilities, muscle weakness, or age-related limitations. Although powered wheelchairs reduce physical strain, they typically rely on a fixed control method such as a joystick. This restricts usability for individuals who cannot effectively move their hands or who experience tremors, paralysis, or neurological conditions affecting fine motor control.

- Another significant problem with current wheelchair systems is the lack of integrated safety mechanisms. Falls, tilting accidents, and collisions are common among wheelchair users, especially in outdoor environments with uneven roads or indoor spaces with obstacles. Many low-cost or traditional wheelchairs do not include fall detection, obstacle avoidance, or emergency alert features. This increases the risk of injury, especially when the user is alone or unable to call for help. Real-time location tracking is another missing element, making it difficult for caregivers to locate users during emergencies or unexpected incidents.
- Additionally, a review of existing research reveals that most systems only offer partial solutions, focusing on a single feature such as voice control, Bluetooth control, or basic obstacle detection. There is a clear lack of unified systems that integrate multiple control methods—such as button control, voice commands, gestures, and joystick input—along with a complete safety suite and GPS tracking. High-quality commercial smart wheelchairs that include advanced features are extremely expensive and inaccessible for most users. Therefore, the key problem is the absence of an affordable, multi-functional, and safety-enhanced smart wheelchair that offers flexible control options and real-time monitoring using modern embedded technologies.

2.3 Objectives

- The primary objective of this project is to design and develop a comprehensive smart electric wheelchair that addresses the limitations identified in existing systems. The system aims to provide a multi-control solution, allowing users to operate the wheelchair through a smartphone application, which includes button-based navigation, voice control, and gesture-based control. In addition to mobile-based control, a physical joystick is included to ensure traditional manual input is always available, even when the smartphone is not in use. This multi-modal control approach increases accessibility for users with varying degrees of physical ability.

- Another important objective is to enhance the safety of wheelchair users by integrating fall detection, obstacle avoidance, and an emergency stop mechanism. Using the MPU6050 sensor, the system continuously monitors tilt angles and sudden accelerations to detect potential falls or instability. The ultrasonic sensor provides real-time distance measurement to detect obstacles in front of the wheelchair and prevent collisions. An emergency button is included to allow immediate shutdown of the motors and activation of the alarm system, ensuring user safety during hazardous situations.

- Real-time GPS tracking is also an essential objective of the project. The integration of a GPS

CHAPTER 3

METHODOLOGY

METHODOLOGY

- The development of the "ESP32-Based Smart Electric Wheelchair" followed a structured methodology involving requirement analysis, system design, hardware development, firmware programming, safety integration, and testing. The goal was to build a reliable, low-cost, and user-friendly mobility system with multi-mode control and real-time safety features.
- First, user needs were analyzed, identifying that joystick-only control was insufficient for users with limited hand movement. Therefore, four control modes were implemented: mobile app buttons, in-app voice commands, in-app gesture control, and a hardware joystick. Safety requirements such as fall detection, obstacle avoidance, emergency stop, and GPS-based location reporting were also defined.
- Next, the system architecture was designed using the ESP32 as the central controller due to its Bluetooth capability, PWM outputs, multiple ADC channels, and dual-core processing. The design integrated motor drivers, sensors, GPS, emergency button, buzzer, LED, and power modules to ensure smooth communication and real-time performance.
- Hardware implementation connected BTS7960 motor drivers for high-torque motors, joystick inputs to the ESP32 ADC pins, MPU6050 via I²C for fall detection, ultrasonic sensor for obstacle sensing, and NEO-6M GPS via UART. A 12V battery and buck converter powered the motors and electronics. An emergency button, buzzer, and LED were added for user safety.
- Firmware was developed in Arduino IDE using C/C++ and required libraries. Non-blocking code enabled simultaneous Bluetooth control, joystick inputs, fall monitoring, obstacle detection, and GPS updates. PWM-based differential motor control ensured smooth movement. Algorithms were created for tilt-based fall detection, automatic stopping, alarm activation, and GPS alerting.
- Safety mechanisms included emergency stop interrupts, timeout-based auto-stop for lost control signals, and continuous ultrasonic distance checking.
- Finally, the system was tested on multiple surfaces and scenarios. All modules—Bluetooth, joystick, fall detection, obstacle avoidance, and GPS—were individually and collectively validated. After adjusting sensor thresholds and motor speeds, the system achieved stable performance and fulfilled all project objectives.

3.1 Proposed System

The proposed system is a fully integrated smart electric wheelchair designed to enhance the mobility, safety, and independence of users with physical disabilities. The system uses the ESP32 microcontroller as the core computational unit to manage inputs, process sensor data, control motors, and communicate with the user through a Bluetooth-based mobile application. The key objective of the proposed system is to provide a flexible multi-mode operating interface that allows the user to control the wheelchair through mobile app buttons, voice commands, touch based gesture inputs, or a physical joystick. These control modes ensure adaptability to the user's physical capabilities and preferences. Additionally, the system includes essential safety features such as fall detection, obstacle avoidance, and a hardware emergency stop button, all of which operate in real-time to protect the user during navigation. The proposed design is intended to be compact, reliable, low-cost, and suitable

for integration with standard wheelchair hardware, making it an effective assistive technology solution.

The proposed system is a **Smart Electric Wheelchair** controlled using four different modes:

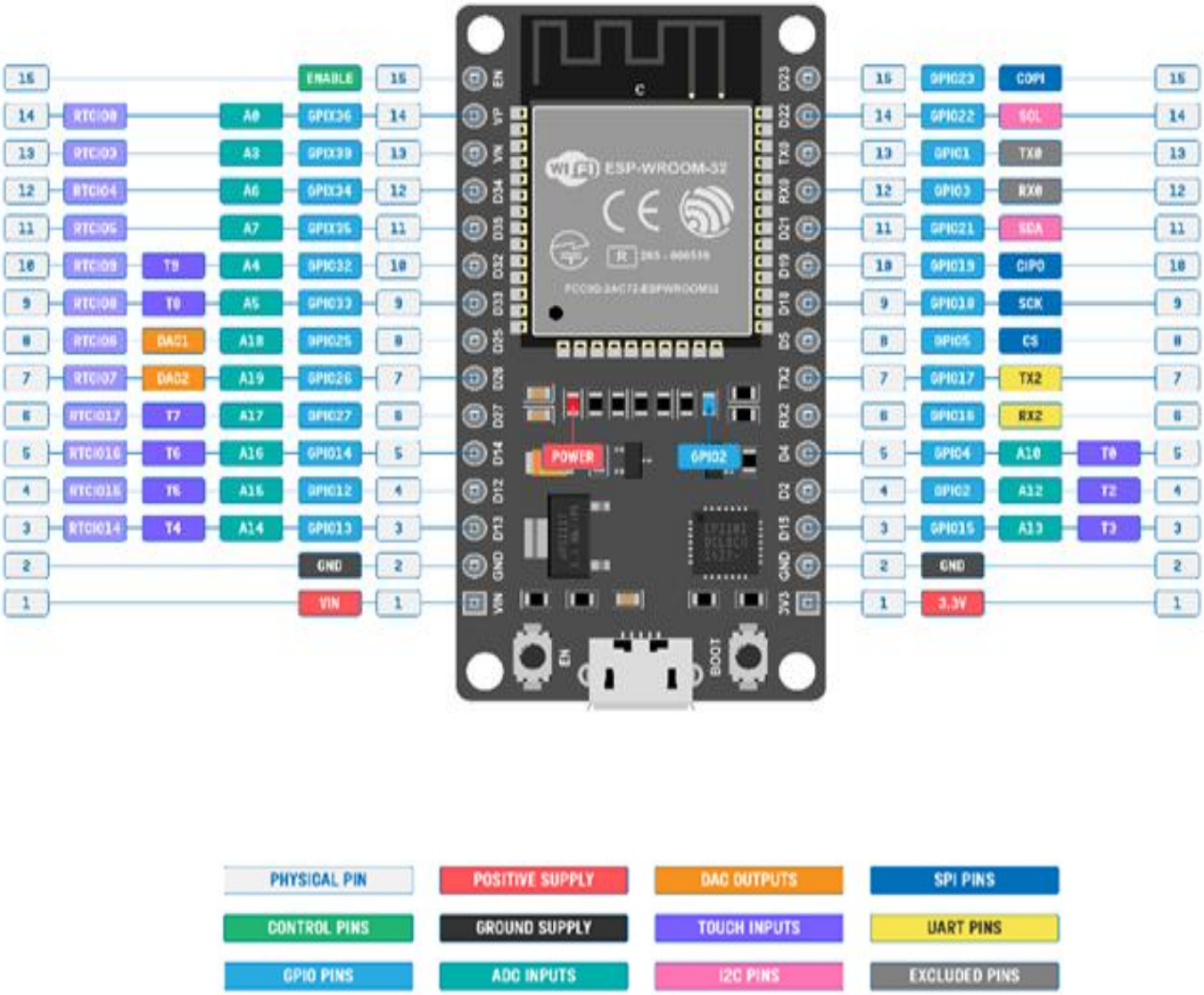
- Mobile App Button Control
- Voice Control (Inside the App)
- Hand-Gesture Control (Inside the App)
- Physical Joystick Control

The wheelchair integrates multiple safety features including **fall detection** (MPU6050), **obstacle detection** (ultrasonic sensor), an **emergency stop button**, and **GPS-based location tracking**. The central controller, **ESP32 Devkit V1**, processes Bluetooth commands, reads sensors, controls the motors, and communicates GPS data. Motor movement is handled through **BTS7960 high power motor drivers**, allowing stable and smooth control of the left and right wheelchair motors. The system ensures reliable mobility, real-time safety monitoring, and flexible control for users with different physical needs.

3.2 Components Used

- ESP32 Devkit V1
- BTS7960 Motor Drivers (x2)
- DC Wheelchair Motors
- Mobile App
- Physical Joystick
- MPU6050 Sensor
- Ultrasonic Sensor (HC-SR04)
- GPS Module (NEO-6M)
- Emergency Stop Button
- Buzzer
- LED Indicator.
- Power System
- Jumper wires
- Mini Bread board

3.2.1 ESP32 Devkit V1 (Microcontroller)



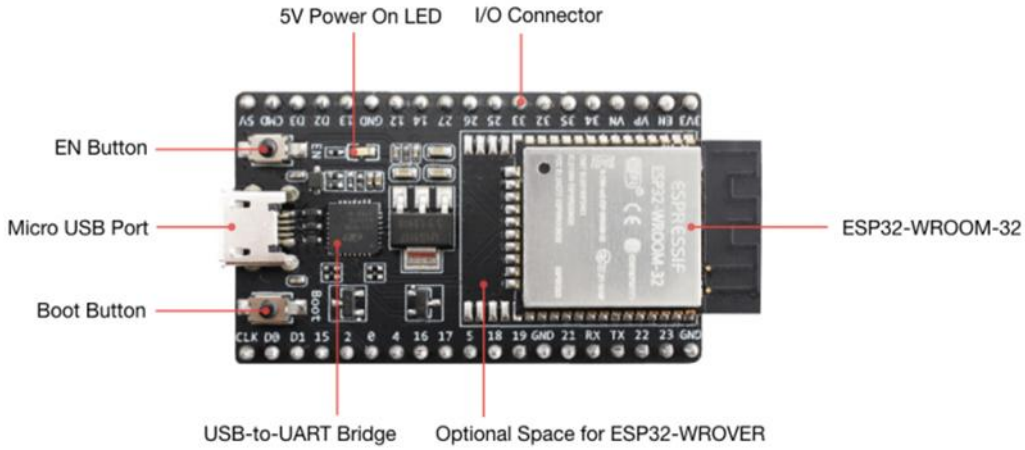


Figure 3.2.1 ESP32 Development Board

- The ESP32 Devkit V1 is the central microcontroller of the smart electric wheelchair, acting as the complete "brain" of the system. It features a dual-core Tensilica processor, built-in Wi-Fi and Bluetooth (Classic + BLE), multiple ADC inputs, PWM outputs, UART/I²C/SPI communication ports, and high processing speed—making it ideal for real-time data handling, multi-sensor integration, and simultaneous control tasks.
- In this project, the ESP32 processes all user inputs, including mobile app button commands, voice commands, and gesture signals, received via Bluetooth. It also reads analog joystick values to allow manual wheelchair operation. Additionally, it continuously collects data from the MPU6050 accelerometer/gyroscope for fall detection and from the ultrasonic sensor for front obstacle measurement.
- Based on all incoming inputs and sensor readings, the ESP32 decides how the wheelchair should move by generating precise PWM signals to the BTS7960 high-power motor drivers, controlling the speed and rotation of the left and right motors. It also manages the buzzer, LED status indicator, speed modes (Boost/Precision), emergency stop logic, and GPS data received via UART communication.
- The ESP32 ensures multi-tasking and parallel operation, meaning Bluetooth input, joystick control, obstacle detection, fall monitoring, and GPS updates all run smoothly without interfering with each other. Its dual-core architecture allows one core to handle motor control and sensor reading while the other manages communication and safety logic.
- The microcontroller is also responsible for safety actions, such as stopping motors during fall detection, sending emergency GPS coordinates, activating alarm patterns, and handling time-out conditions when user control becomes inactive. It ensures the wheelchair behaves safely even in uncertain situations.
- With its support for high-speed data processing and wireless communication, the ESP32 enables seamless integration of smart features such as voice control, app control, gesture detection, GPS reporting, and real-time monitoring—making the wheelchair intelligent, adaptive, and user-friendly.

3.2.2 BTS7960 Motor Driver

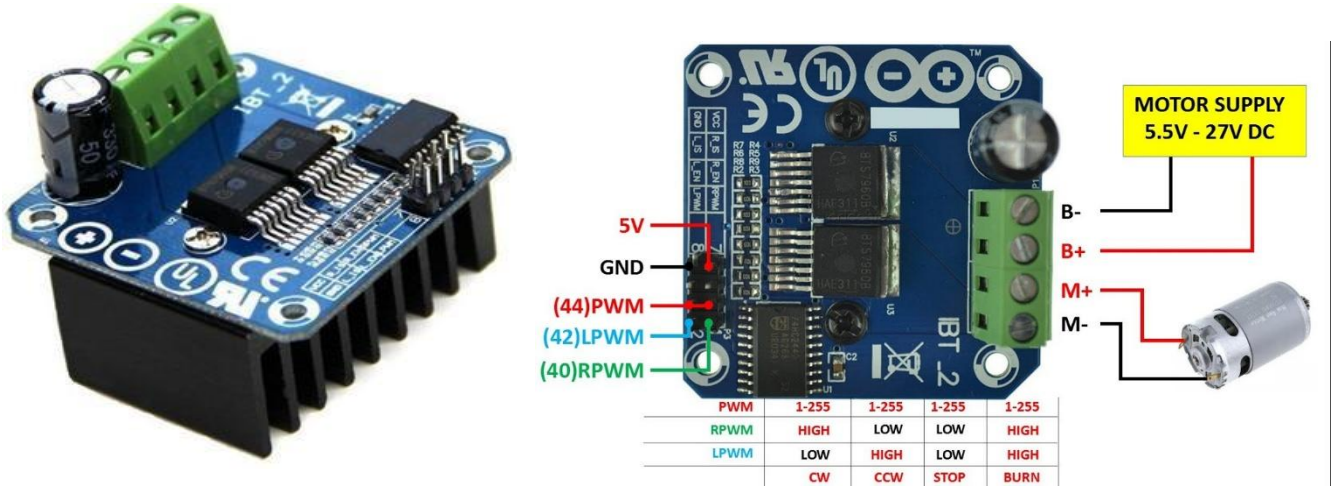


Figure 3.2.2 BTS7960 Motor Driver

The BTS7960 is a high-power dual H-bridge motor driver capable of handling currents up to 43A, making it highly suitable for driving wheelchair motors that demand high torque and continuous load handling. Compared to traditional low-power drivers like L298N or L293D, the BTS7960 provides superior power efficiency, low heat generation, smoother PWM response, and reliable performance under heavy loads.

Why It Is Used in the Wheelchair System

The BTS7960 was specifically selected because it offers:

- High current handling (up to 43A) suitable for real wheelchair motors
- Smooth and stable motor control using high-frequency PWM
- Safe operation under heavy loads thanks to built-in protection features
- Bidirectional motor control for both forward and reverse movement
- Low voltage drop & high efficiency compared to older drivers
- Better torque delivery, essential for uneven surfaces or ramps

Low-power drivers such as L298N cannot deliver the required current and typically overheat or fail under wheelchair load, making the BTS7960 the correct choice for this application.

Role of BTS7960 in the Smart Wheelchair

In this project, the BTS7960 motor driver performs several essential functions:

- Drives the left and right wheelchair motors with high current output.
- Receives PWM signals from the ESP32 to control motor speed precisely.
- Enable smooth forward, reverse, and stop control of each motor.
- Supports differential steering, allowing the wheelchair to turn smoothly.
- Provides controlled acceleration and deceleration, increasing user safety.
- Protects the system with built-in overcurrent, overheating, and short-circuit protection.
- Ensures durability and stable operation during continuous long-term us

3.2.3 Battery (12V Lead-Acid/Li-ion)



Figure 3.2.3 Wheelchair Battery

The 12V battery is the main power source that provides energy to both motors and the electronic circuitry. Wheelchair motors require high current, so a reliable battery is essential for consistent performance and long usage time. Lead-acid and lithium-ion batteries are commonly used due to their high capacity and durability.

The battery functions by delivering sufficient voltage and current to run high-power motors, the motor drivers, and supporting electronics. A buck converter is used to regulate power for sensitive components like the ESP32 and sensors, ensuring stable and noise-free operation.

In this project, the 12V battery powers the motors and the entire system. Its ability to provide stable current ensures smooth and uninterrupted wheelchair movement

3.2.4 Ultrasonic Sensor (HC-SR04)

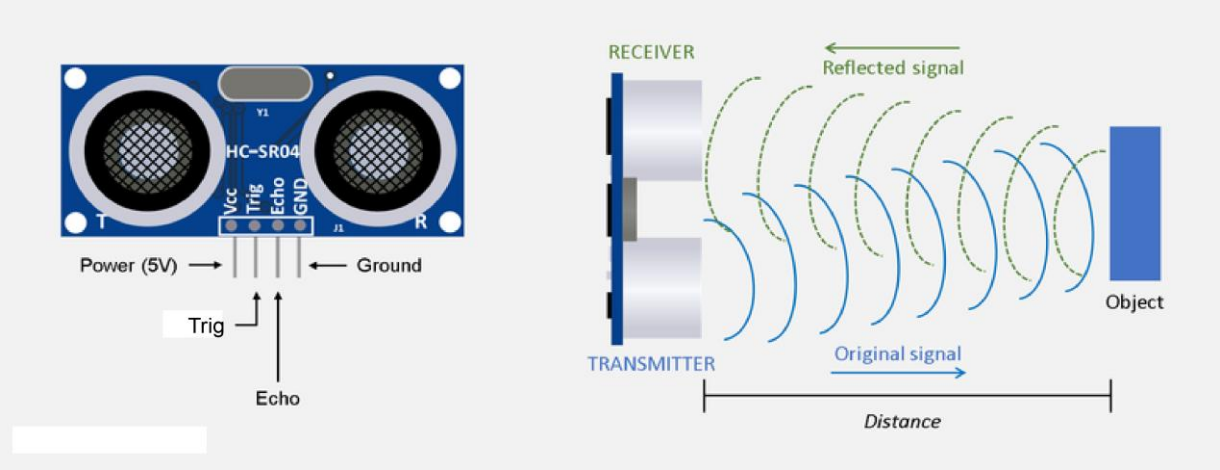


Figure 3.2.4 Ultrasonic Distance Sensor

The ultrasonic sensor is used to measure the distance between the wheelchair and any obstacle in its front path. It works by emitting ultrasonic waves from the transmitter and detecting the returning echo using the receiver. The time interval between sending and receiving the pulse is

converted into distance. This sensor provides reliable non-contact measurement and is widely used in robotics and automation applications for detecting nearby objects.

The primary function of the ultrasonic sensor is to prevent collisions. It continuously scans the area ahead of the wheelchair and provides distance feedback to the ESP32. When an obstacle is detected within the preset safe distance, the system immediately stops the motors and alerts the user through audible signals from the buzzer. This provides an additional layer of safety, especially in narrow spaces, crowded environments, or low-visibility conditions.

In the smart wheelchair project, the ultrasonic sensor enables obstacle avoidance functionality. It ensures that the wheelchair does not hit walls, furniture, or other objects during movement. The ESP32 uses distance data from the sensor to make real-time decisions about stopping, slowing down, or continuing movement. This improves navigation safety and user confidence.

3.2.5 GPS Module (NEO-6M)

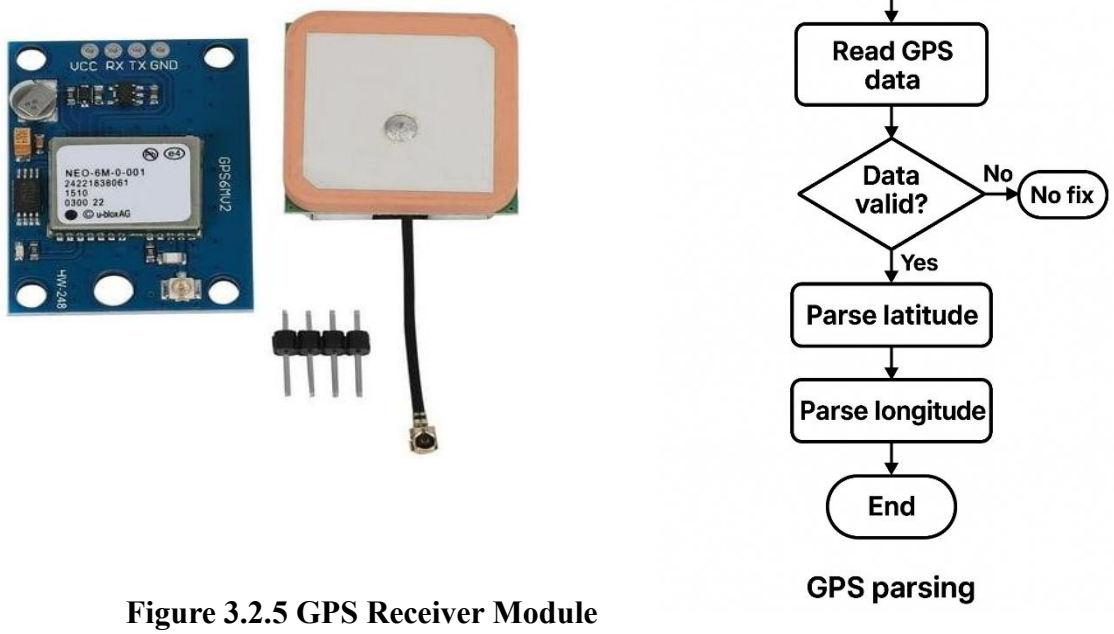


Figure 3.2.5 GPS Receiver Module

The GPS module is responsible for acquiring the real-time geographic location of the wheelchair. It receives signals from multiple satellites and calculates accurate latitude and longitude coordinates. The module communicates with the ESP32 via UART, delivering continuous NMEA sentence data. It also provides metadata such as satellite fix status, signal quality, and timestamp.

The module functions by triangulating the wheelchair's position using satellite signals. The ESP32 processes the data using a GPS parsing library and extracts the required coordinates. These coordinates can be transmitted to the mobile application at regular intervals or during emergencies. This allows caregivers or other monitoring personnel to track the wheelchair's location.

In this project, the GPS module enhances safety and monitoring. During normal operation, it periodically sends location updates. In the event of a fall or emergency button press, it

immediately reports the wheelchair's exact location. This ensures that help can reach the user quickly during critical situations.

3.2.6 DC Wheelchair Motors



Figure 3.2.6 Motor driver

The wheelchair motors are robust 12V DC motors designed to handle high torque and heavy weight, making them suitable for mobility and load-bearing applications. These motors are capable of propelling the wheelchair with a user seated on it, providing stable movement on both flat and inclined surfaces. The motors are specifically built for wheelchairs and similar assistive devices, where reliable and continuous performance is essential.

The motors receive controlled power from the BTS7960 motor drivers, which adjust their speed and direction. This allows the system to perform forward, reverse, left, and right movements. The motors also support differential steering, where varying the speed of the left and right motors results in smooth turning maneuverability. These motors are an essential mechanical component that physically converts electrical control signals into actual motion of the wheelchair.

In the project, the motors serve as the primary movement mechanism of the entire system. Their torque and reliability ensure that the wheelchair can move safely, carry user load, and operate in diverse environments such as indoor floors, outdoor pavements, or ramps. Their compatibility with 12V power supply and high-current motor drivers make them ideal for the smart wheelchair's mobility framework.

3.2.7 Mecanum Wheels



Figure 3.2.7 Mecanum Wheel Assembly

Mecanum wheels are omni-directional wheels designed with 45° angled rollers around their circumference, allowing movement in any direction without rotating the wheelchair's body.

Their unique roller arrangement enables forward/backward motion, sideways sliding, diagonal movement, and zero-radius turning—capabilities not possible with normal wheels.

The wheels operate using vector force generation: as each wheel spins, the angled rollers redirect part of the force sideways. By varying the speed and direction of all four wheels, the wheelchair can move smoothly in any direction. For example, rotating opposite diagonal wheels in the same direction produces lateral (sideways) motion.

In this project, mecanum wheels greatly improve maneuverability, especially in tight indoor spaces, narrow hallways, and crowded environments. Paired with high-power DC motors and BTS7960 motor drivers, they allow precise control using joystick, voice commands, gesture control, and mobile app button inputs. Their strong construction ensures stability and durability during continuous use.

Additionally, mecanum wheels reduce the need for large turning spaces and enable highly precise alignment in medical or assisted-living environments. They offer improved user comfort by delivering smoother directional transitions. Mecanum wheels also help minimize collisions since the wheelchair can quickly shift sideways to avoid obstacles. Their mechanical design supports better navigation on slightly uneven surfaces. Overall, they significantly enhance the wheelchair's adaptability and performance in both indoor and controlled outdoor settings.

3.2.8 Joystick Module

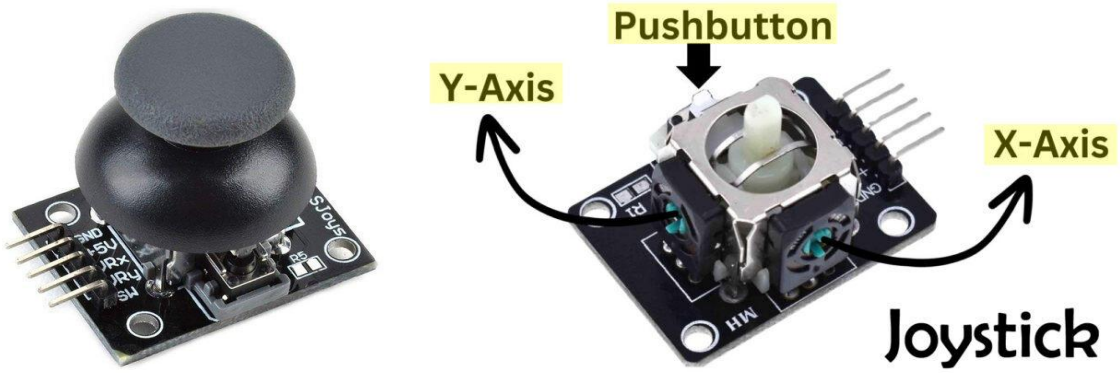


Figure 3.2.8 Two-Axis Analog Joystick

The joystick module is a two-axis analog input device that allows the user to manually control the movement of the wheelchair. It consists of two potentiometers that measure displacement along the X and Y axes and convert them into varying voltage signals. The ESP32 reads these analog voltages through its ADC pins and converts them into PWM motor control commands. The joystick also includes a built-in push button that can be used for additional functions such as toggling emergency modes.

The joystick functions as a proportional control device, meaning the degree of tilt determines the speed and direction of the movement. Slight movements correspond to slow motion, while larger tilts generate higher speeds. This provides fine-grained control and a more natural driving experience compared to fixed-speed systems. The wheelchair also calibrates the joystick at startup to determine the neutral center position, ensuring stable manual operation.

In the project, the joystick serves as an essential fallback or primary control method for users who prefer manual operation. It is especially useful when the smartphone is unavailable, when Bluetooth connectivity is lost, or when the user requires precise, real-time manual control. The

joystick adds significant flexibility to the system by enabling both digital and analog control modes.

3.2.9 MPU6050 (Accelerometer & Gyroscope Sensor)

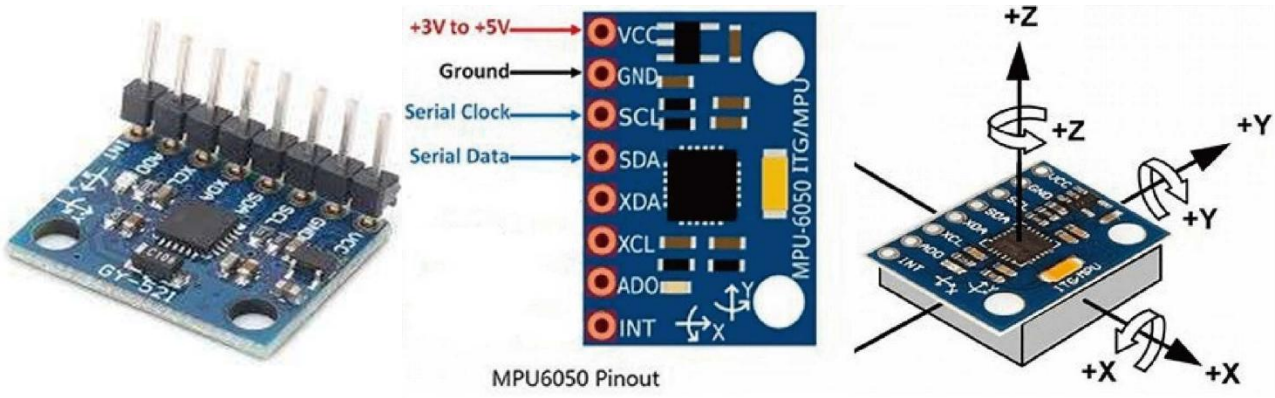


Figure 3.2.9 MPU6050 Sensor Module

The MPU6050 is a combined 3-axis accelerometer and 3-axis gyroscope sensor used to detect motion, orientation, and tilt. It communicates with the ESP32 through the I²C interface, allowing continuous monitoring of acceleration and angular velocity. This sensor is capable of measuring changes in position, speed, tilt angle, and orientation, making it suitable for detecting falls or abnormal movement patterns. Its integrated Digital Motion Processor (DMP) enhances accuracy and stability by processing raw sensor data internally before sending the results to the microcontroller.

The MPU6050 functions by sensing variations in tilt and sudden motion. Accelerometer data provides information about linear acceleration and inclination, while gyroscope data measures rotation and angular movement. By combining these readings, the ESP32 can determine whether the wheelchair has tilted dangerously, lost stability, or experienced sudden impact. These calculations are performed in real time to ensure a quick system response.

In this project, the MPU6050 module is used for fall detection. When it detects a tilt angle greater than the safety threshold or senses free-fall conditions, it immediately triggers an emergency response. The ESP32 stops the motors, activates the alarm system, and sends the wheelchair's GPS location to the mobile application. This sensor ensures user safety by preventing accidents and providing immediate notification during emergencies.

3.2.10 Jumper Wires

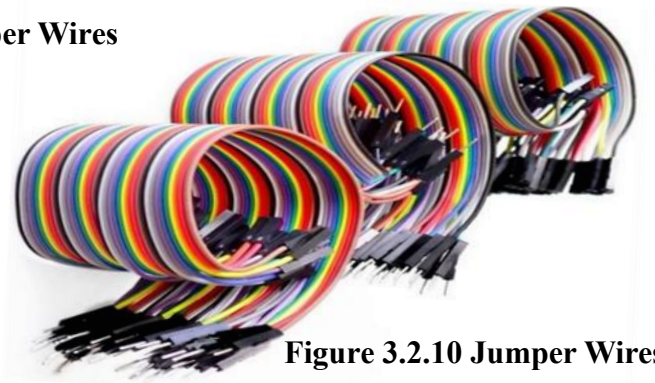


Figure 3.2.10 Jumper Wires Set

Jumper wires are flexible electrical connectors used to interconnect various components of the circuit without soldering. They allow quick prototyping and provide ease of modification. These

wires come in male-to-male, female-to-female, and male-to-female configurations to fit different pin types.

The jumper wires function by establishing electrical connections between the ESP32, sensors, motor drivers, and other electronic modules. They ensure proper signal transmission and power distribution across the system.

In this project, jumper wires are used to connect all sensors, modules, and control components to the ESP32. They make the setup modular, secure, and easy to maintain.

3.3App Home Interface (Remote Control Layout)



Figure 3.3.1 App Home Interface (Remote Control Layout)

The mobile application is a Bluetooth-based control interface developed to provide flexible and convenient operation of the smart wheelchair. The app includes three primary control modes: traditional button-based control, voice command control using in-app speech processing, and hand gesture control using touch-based swipe gestures. This variety of control options ensures accessibility for users with different physical abilities and mobility challenges.

The application communicates wirelessly with the ESP32 microcontroller using Bluetooth Serial communication. When the user interacts with the app—pressing directional buttons, giving voice commands, or performing gestures—the app sends encoded characters (e.g., U, D, L, R, S) to the ESP32. The microcontroller interprets these commands and translates them into movement actions. The mobile app also provides additional functions such as speed mode selection, emergency alert reception, and GPS location updates.

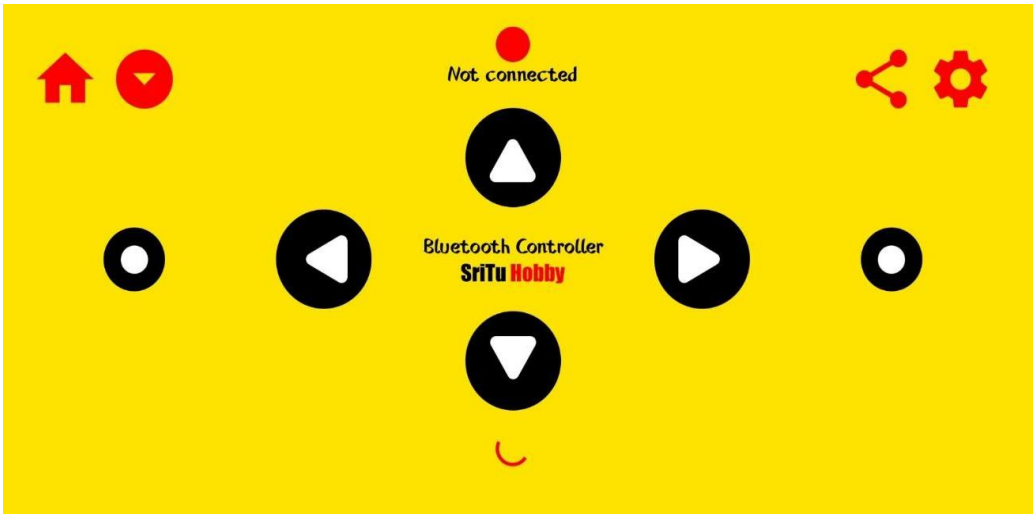
In the context of the smart wheelchair, the mobile application acts as the primary user interface. It allows users who cannot operate physical joysticks to control the wheelchair with ease. The integration of voice and gesture control significantly enhances accessibility, making the wheelchair more inclusive for users with severe hand movement limitations, visual impairment, or neurological disorders. The application thus forms an essential part of the human-machine interaction layer of the system.

This screen displays the main control layout with large, easy-to-tap directional buttons.
It is designed for **fast response, clarity, and accessibility** for users with limited hand mobility.



3.3.2 Directional Control Screen

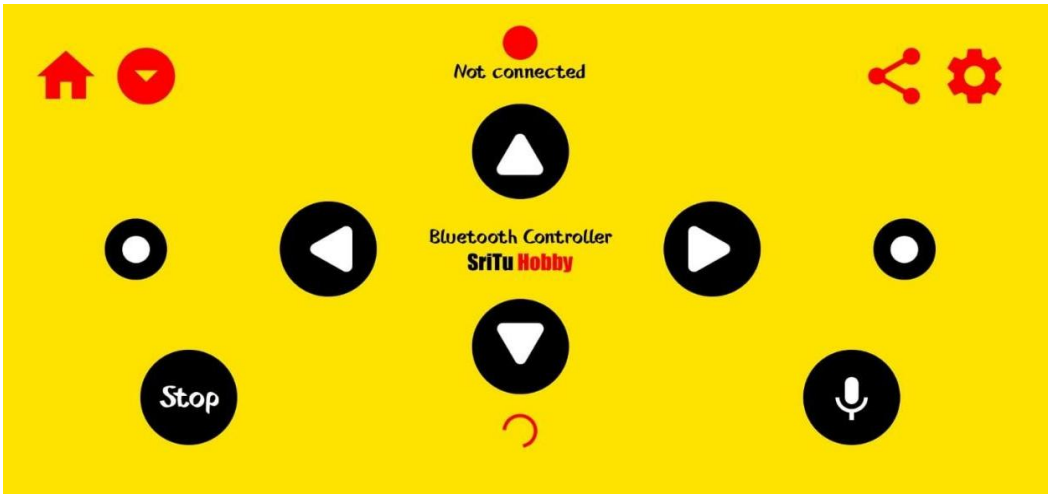
This screen provides **four-direction movement** (forward, backward, left, right).
It includes Bluetooth status indicators and navigation icons for switching modes.



3.3.3 Expanded Control Screen with Stop & Voice Commands

This interface extends the directional controls by adding:

- **STOP button** for instant halt
- **Voice command button** to activate voice-based control
- **Bluetooth connection status**
- **Settings & Share icons**



3.3.4 Description of App Features

- **Directional Buttons:** Forward, backward, left, right.
- **Voice Input Button:** Activates microphone-based movement control.
- **Stop Button:** Immediate stop signal to ESP32 for safety.
- **Bluetooth Status Indicator:** Shows connection/disconnection.
- **Navigation Icons:** Home, settings, share, and mode switching.
- **Clean UI:** Yellow-black theme for visibility and ergonomics.
- **Error-free Communication:** Sends encoded characters ('U', 'D', 'L', 'R', etc.) to ESP32.

3.3.5 Role in the Wheelchair System

The mobile app plays an essential role in:

- Wireless control of the smart wheelchair
- Voice-activated movement
- Gesture-based input (depends on mode)
- Real-time command transmission
- Emergency stop control
- Monitoring Bluetooth status

This makes the wheelchair accessible even for users with **minimal hand movement**.

3.3.6 MOBILE APPLICATION SCREENSHOTS

The following screenshots show the user interface of the Bluetooth-based mobile application used to control the Smart Electric Wheelchair.
This application provides **button control, directional navigation, voice command activation, and Bluetooth connectivity status**, enabling the user to operate the wheelchair intuitively and safely.

3.4 Software and Code Explanation

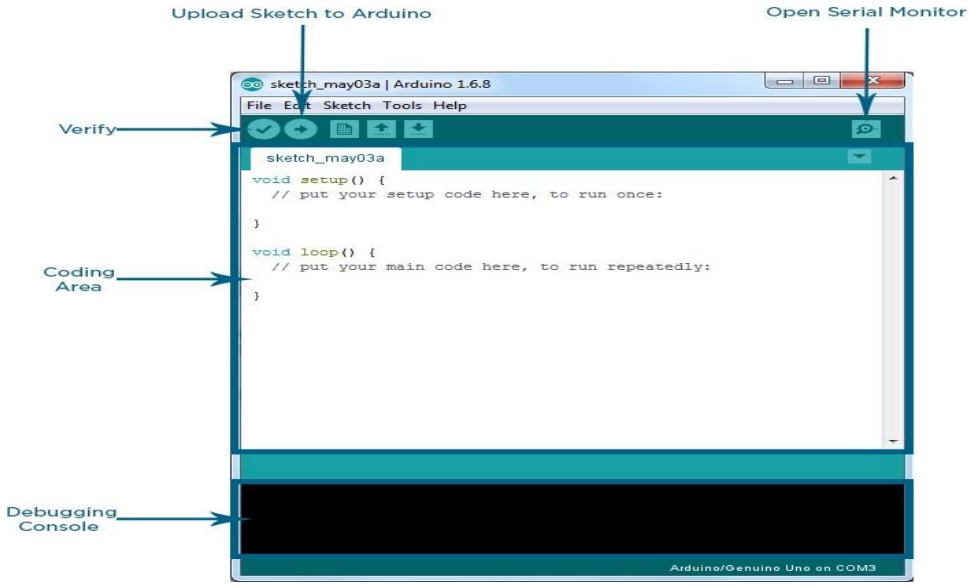


Figure 3.4 Software and Code Explanation

The software for the smart wheelchair was developed using the Arduino IDE with C/C++ programming. The code is structured into modules to handle Bluetooth communication, joystick input processing, sensor readings, motor control algorithms, GPS parsing, and emergency handling. At startup, the ESP32 initializes the UART, I²C, and Bluetooth modules. The Bluetooth handler listens for incoming characters and translates them into movement commands such as forward, backward, left, right, boost mode, or precision mode. The joystick handler reads analog values and maps them to differential motor speeds.

The fall detection algorithm continuously reads accelerometer and gyroscope data from the MPU6050 and calculates tilt angles. If dangerous tilt conditions are detected, the alarm system is triggered. The ultrasonic subsystem uses pulse measurement to calculate distance and triggers collision-avoidance routines. GPS data is read through UART, processed using the TinyGPSPlus library, and transmitted via Bluetooth to the mobile app. The emergency button, buzzer, and LED are managed through interrupt-based or periodic checks, ensuring that safety mechanisms respond instantly. The entire code follows non-blocking architecture to ensure smooth and real-time operation of all modules simultaneously.

3.5 Block Diagram Explanation

The block diagram of the system illustrates the architecture of the smart wheelchair and the interconnections between its various modules. At the center of the diagram is the ESP32 microcontroller, responsible for processing all inputs and controlling outputs. The mobile application communicates with the ESP32 through Bluetooth, enabling button-based, voice-based, and gesture-based control signals to be transmitted wirelessly. The joystick is connected directly to the analog input pins of the ESP32, providing an alternative manual control interface.

The ESP32 interfaces with the BTS7960 motor drivers using PWM signals, enabling precise control over the speed and direction of the left and right motors.

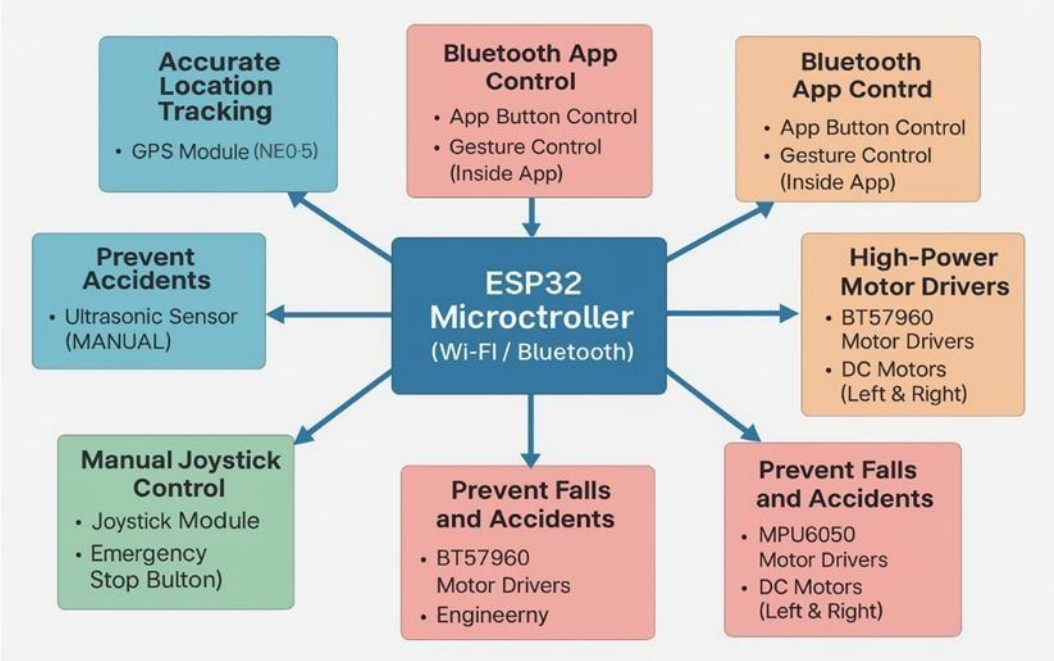
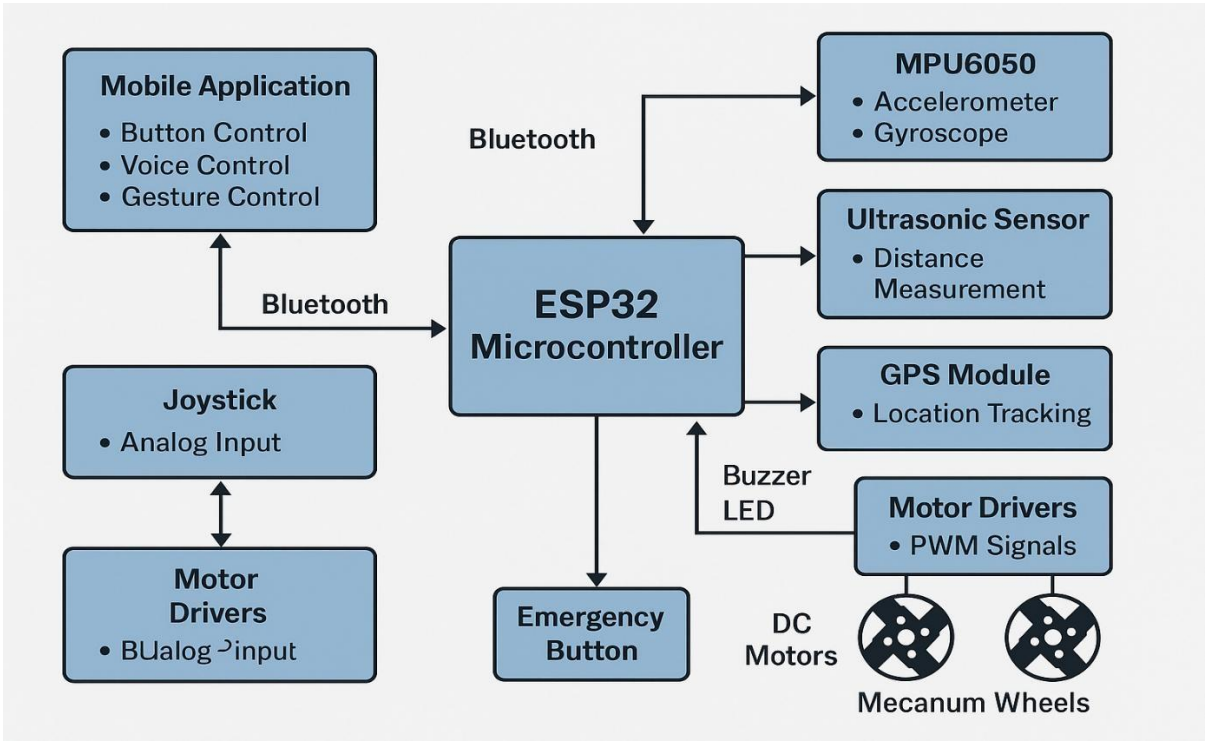
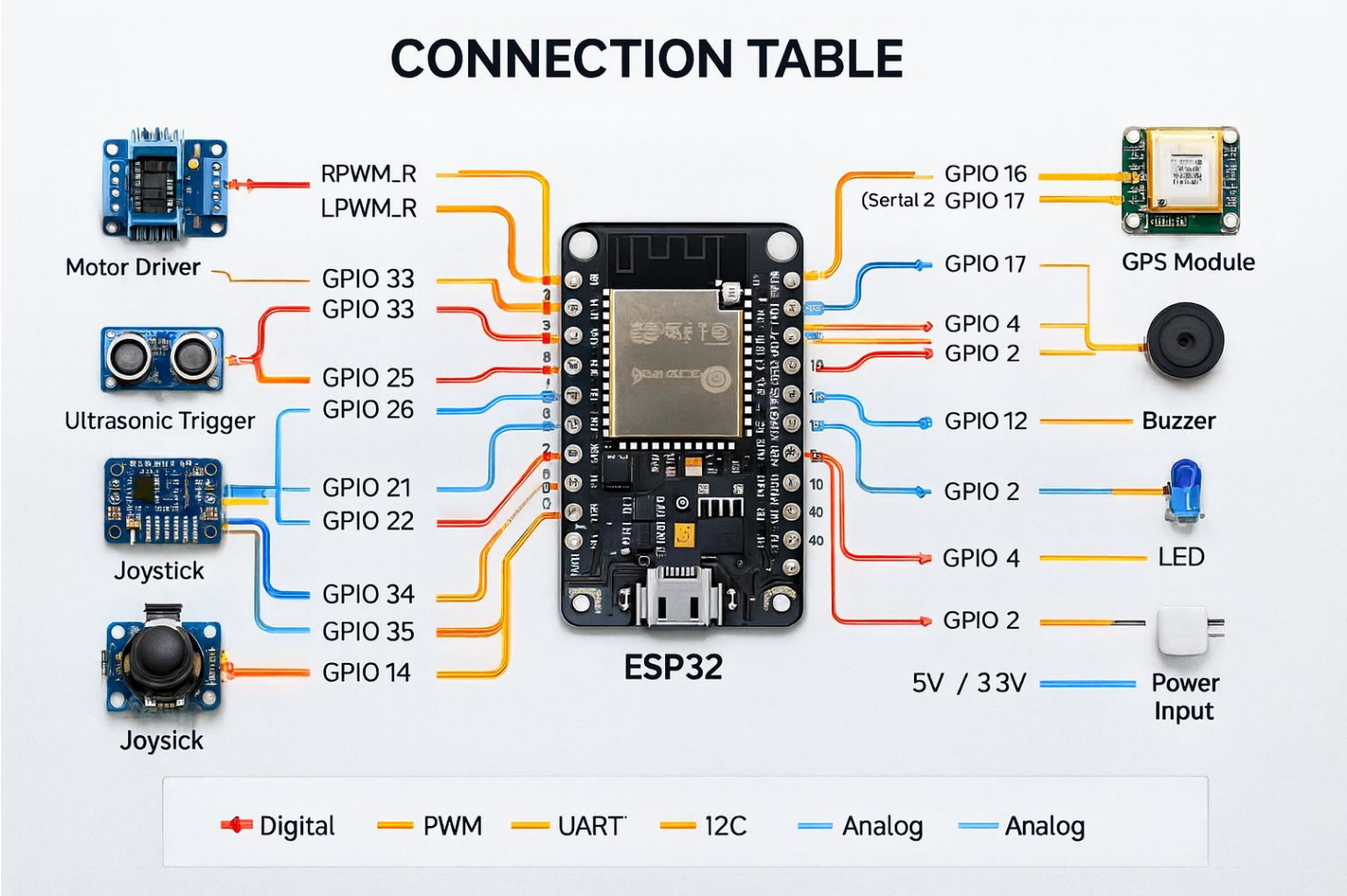


Figure 3.5.1 Block Diagram Explanation

the MPU6050 module communicates with the ESP32 through the I²C interface for fall detection, while the ultrasonic sensor uses trigger and echo pins for obstacle measurement. The GPS module uses UART communication to feed coordinates to the ESP32, which can then forward them to the mobile app during emergencies. The emergency stop button, buzzer, and LED indicator are connected to digital input and output pins of the ESP32 to handle user alerts and system status. The block diagram shows the flow of data from sensors and user inputs to the ESP32 and the corresponding motor and alert outputs that ensure safe and controlled wheelchair operation.



3.6 Circuit Diagram



3.6 Circuit Diagram Explanation

The circuit diagram represents the electrical connectivity of each component in detail. The ESP32 Devkit V1 is the primary controller with connections to the BTS7960 motor drivers for controlling the left and right motors. PWM pins from the ESP32 are connected to RPWM and LPWM pins of each motor driver. The motor drivers are powered by the 12V battery, while the motors are connected directly to the high-current output terminals of the motor drivers. The joystick is connected to the analog pins (34 and 35) to read directional analog values, and the joystick switch is connected to a digital pin for emergency toggling.

The MPU6050 module connects to the ESP32 through the SDA and SCL pins for I²C communication, while the ultrasonic sensor is connected using digital trigger and echo pins. The GPS module connects through UART pins (RX and TX) to enable continuous data reception. The emergency stop button is wired to a digital pin with an internal pull-up resistor, allowing the system to detect button presses reliably. The buzzer and LED are connected through output pins and provide audio and visual alerts. A buck converter provides regulated 5V to the ESP32 and all sensors, while the motors draw power directly from the high-current battery. This circuit layout ensures a stable and safe electrical system capable of driving the wheelchair's mechanical load.

3.7 WORKING

The ESP32-Based Smart Electric Wheelchair works by combining multi-mode user control, real-time safety monitoring, and synchronized motor operation to deliver smooth and reliable mobility for the user.

1. System Initialization

When the wheelchair is powered ON, the ESP32 begins by initializing all the communication modules:

- **Bluetooth** – for app control, voice control, and gesture inputs
- **I²C** – for reading data from the MPU6050 (accelerometer + gyroscope)
- **UART** – for the GPS module
- **Digital I/O** – for ultrasonic sensor, buzzer, emergency stop button
- **Analog Input** – for joystick X/Y axis signals

The joystick automatically calibrates at startup to find the neutral (center) position so that manual control remains accurate and smooth.

Once all sensors and modules are ready, the ESP32 switches to active mode.

2. Multi-Mode Control Operation

The wheelchair supports **four different control modes**, giving users flexibility based on their comfort and mobility needs:

a) Mobile App Button Control (Bluetooth)

The app sends encoded characters like ‘F’, ‘B’, ‘L’, ‘R’ for forward, backward, left, and right. ESP32 reads these commands, processes them, and generates PWM signals to control motor speed and direction.

b) Voice Control

Voice commands are converted into Bluetooth instructions by the mobile app.
These are then translated by the ESP32 into motor actions.

c) Gesture Control (Mobile Motion Sensor)

The app detects phone tilt (left/right/forward/backward) and sends corresponding commands to the ESP32, allowing motion by hand gestures.

d) Manual Joystick Control

The joystick gives proportional analog values.
The ESP32 maps these values into motor speeds, enabling:

- Smooth acceleration
- Precise turning

- Zero-delay manual control

3. Coordinated Motor Operation

The wheelchair uses **BTS7960 high-power motor drivers**, capable of handling heavy loads. The ESP32 calculates the required movement based on input and sends appropriate PWM signals to each motor.

With **mecanum wheels**, the wheelchair achieves:

- Forward / backward
- Left / right
- Diagonal
- Rotation on the spot
- 360° omni-directional movement

4. Real-Time Safety Monitoring

Safety is monitored continuously while the wheelchair is moving.

a) Obstacle Detection (Ultrasonic Sensor)

- Measures front distance in real time
- If an obstacle is too close, the system:
    - Immediately stops the motors
    - Activates the buzzer
    - Prevents collisions even if the user still presses forward

b) Fall / Tilt Detection (MPU6050)

The MPU6050 monitors:

- Tilt angle
- Sudden jerks
- Free-fall-like movement

If the tilt crosses a safe limit or detects unusual movement:

- Motors stop instantly
- High-frequency alarm activates
- GPS location is sent to the mobile app for help

c) Emergency Stop Button

This is a **manual override**:

- Cuts off all PWM signals
- Stops the motors immediately
- Triggers buzzer
- Sends real-time GPS coordinates
- Works even if ESP32 is receiving other commands

5. GPS Tracking and Emergency Communication

The GPS module continuously updates location data.
The ESP32:

- Sends periodic location updates through the app
- Sends instant coordinates during emergencies or falls
- Helps caregivers track the wheelchair in real time

6. Summary (Human-Friendly)

In simple terms, the ESP32 acts like the *brain* of the wheelchair.
It reads commands from the app or joystick, controls the motors through BTS7960 drivers, and keeps checking all safety sensors. If it notices anything unsafe—such as an obstacle, a fall, or the emergency button being pressed—it immediately stops the wheelchair and alerts the user or caregiver.

The combination of:

- Multi-mode control
- Omni-directional mecanum movement
- Obstacle avoidance
- Tilt/fall detection
- GPS tracking
- Emergency system

3.8 Code

```
 #include <Wire.h>
#include <MPU6050.h>
#include <TinyGPSPlus.h>
#include "BluetoothSerial.h"

// ===== MODULES =====
BluetoothSerial SerialBT;
TinyGPSPlus gps;
HardwareSerial SerialGPS(2);
MPU6050 mpu;

// ===== PINS ===== const int RPWM_R = 32, LPWM_R = 33,
RPWM_L = 25, LPWM_L = 26; const int JOY_X = 34, JOY_Y = 35,
JOY_SW = 14; const int TRIG_PIN = 18, ECHO_PIN = 19; const int
BUZZER_PIN = 4, LED_PIN = 2;

// ===== SPEED SETTINGS =====
int btSpeed = 45;          // Normal Bluetooth
speed int maxSpeed = 120;        // Max speed for
joystick const int BOOST_SPEED = 70; const
int PRECISION_SPEED = 30;

// ===== JOYSTICK CONFIG =====
int centerX = 2048, centerY = 2048;
const int deadzone = 600;
// ===== STATE =====
bool btActive = false, joystickActive = false; bool alarmActive = false,
buttonAlarmActive = false, fallDetected = false; bool blinkState = false;
unsigned long lastBlink = 0; unsigned long lastBTTime = 0,
lastJoyTime = 0; const unsigned long BT_TIMEOUT = 3000,
JOY_TIMEOUT = 800; unsigned long lastGPSread = 0, lastGPSsend =
0, lastFallCheck = 0, lastObstacleCheck = 0; bool lastButtonState =
HIGH;

// ===== SETUP =====
void setup() {
  Serial.begin(115200);
Wire.begin(21, 22);
mpu.initialize();
  SerialBT.begin("ESP32_WHEELCHAIR");
```

```
  SerialGPS.begin(9600, SERIAL_8N1, 16, 17);
  pinMode(RPWM_R, OUTPUT); pinMode(LPWM_R, OUTPUT);
pinMode(RPWM_L, OUTPUT); pinMode(LPWM_L, OUTPUT);
pinMode(TRIG_PIN, OUTPUT); pinMode(ECHO_PIN, INPUT);
pinMode(LED_PIN, OUTPUT); pinMode(BUZZER_PIN, OUTPUT);
pinMode(JOY_SW, INPUT_PULLUP);

  stopAll();

  long sumX = 0, sumY = 0;
  for (int i = 0; i < 30; i++) {
    sumX += analogRead(JOY_X);
sumY += analogRead(JOY_Y);
    delay(10);
  }
  centerX = sumX / 30; centerY = sumY / 30;

  Serial.printf("🎯 Joystick centered: %d,%d\n", centerX, centerY);
  Serial.println("☑ System ready!");
}

// ===== LOOP =====
void loop() {
handleBluetooth();
handleJoystick();
handleButton();
handleGPS();

  if (millis() - lastFallCheck > 500) { detectFall(); lastFallCheck = millis(); }    if (millis()
- lastObstacleCheck > 200) { obstacleCheck(); lastObstacleCheck = millis(); }

  if (btActive && millis() - lastBTTime > BT_TIMEOUT) { btActive = false; stopAll(); }
  if (joystickActive && millis() - lastJoyTime > JOY_TIMEOUT) { joystickActive = false;
stopAll(); }

  if (alarmActive) emergencySirenPattern();

  handleBlinkActivity();
}
```

```
// ===== LED BLINK INDICATOR =====
void handleBlinkActivity() {
bool anyTask =
     btActive ||
joystickActive ||
alarmActive ||
fallDetected ||
     buttonAlarmActive;


  if (anyTask) {     if (millis() -
lastBlink > 400) {
blinkState = !blinkState;
digitalWrite(LED_PIN,
blinkState);
     lastBlink = millis();
    }
  } else {
    digitalWrite(LED_PIN, LOW);
  }
}


// ===== MOTOR CONTROL ===== void
setMotorPWM(int leftPWM, int rightPWM) {
  if (alarmActive) { stopAll(); return; }


  leftPWM = constrain(leftPWM, -maxSpeed, maxSpeed);
rightPWM = constrain(rightPWM, -maxSpeed, maxSpeed);


  if (leftPWM >= 0) { analogWrite(RPWM_L, leftPWM); analogWrite(LPWM_L, 0); }
else { analogWrite(RPWM_L, 0); analogWrite(LPWM_L, abs(leftPWM)); }


  if (rightPWM >= 0) { analogWrite(RPWM_R, rightPWM); analogWrite(LPWM_R, 0);
}    else { analogWrite(RPWM_R, 0); analogWrite(LPWM_R, abs(rightPWM)); } }


void stopAll() {   analogWrite(RPWM_R, 0);
analogWrite(LPWM_R, 0);   analogWrite(RPWM_L,
0); analogWrite(LPWM_L, 0);
}


// ===== JOYSTICK =====
void handleJoystick() {   if
(btActive || alarmActive) return;
```

```
  int x = analogRead(JOY_X) - centerX;
  int y = centerY - analogRead(JOY_Y);

  if (abs(x) < deadzone) x = 0;   if (abs(y) < deadzone) y = 0;   if
(x == 0 && y == 0) { stopAll(); joystickActive = false; return; }

  joystickActive = true; lastJoyTime = millis();

  int pwmLeft = constrain(map(y - x, -2048, 2048, -maxSpeed, maxSpeed), -maxSpeed,
maxSpeed);
  int pwmRight = constrain(map(y + x, -2048, 2048, -maxSpeed, maxSpeed), -maxSpeed,
maxSpeed);

  setMotorPWM(pwmLeft, pwmRight);
}

// ===== BLUETOOTH CONTROL =====
void handleBluetooth() {   while
(SerialBT.available()) {     char c =
toupper(SerialBT.read());     if (c
== '\r' || c == '\n') continue;
    btActive = true; lastBTTime = millis(); joystickActive = false;

    if (alarmActive) { stopAll(); return; }

    switch (c) {       case 'U': setMotorPWM(btSpeed,
btSpeed); break;       case 'D': setMotorPWM(-
btSpeed, -btSpeed); break;       case 'L':
setMotorPWM(-btSpeed, btSpeed); break;       case
'R': setMotorPWM(btSpeed, -btSpeed); break;
      case 'S': stopAll(); break;       case
'G': sendGPSLocation(); break;
case 'Q': setMotorPWM(btSpeed *
0.5, btSpeed); break;       case 'E':
setMotorPWM(btSpeed, btSpeed *
0.5); break;       case 'Z':
setMotorPWM(-btSpeed * 0.5, -
btSpeed); break;       case 'C':
setMotorPWM(-btSpeed, -btSpeed *
0.5); break;

      case 'B': btSpeed = BOOST_SPEED;
```

```
            Serial.println("🚀 Boost Mode ON (70)");
            SerialBT.println("🚀 Boost Mode ON (70)"); break;
case 'P': btSpeed = PRECISION_SPEED;
            Serial.println("🚗 Precision Mode ON (30)");
            SerialBT.println("🚗 Precision Mode ON (30)"); break;


    default:
      Serial.println("❌ Invalid BT command");
      SerialBT.println("❌ Invalid BT command");
      break;
   }
  }
}

// ===== FALL DETECTION =====
void detectFall() {   if
(buttonAlarmActive) return;

  int16_t ax, ay, az;   mpu.getAcceleration(&ax, &ay, &az);
float axg = ax / 16384.0, ayg = ay / 16384.0, azg = az /
16384.0;   float totalAccel = sqrt(axg * axg + ayg * ayg + azg *
azg);   float tiltAngle = acos(azg / totalAccel) * 57.3;

  if ((tiltAngle > 50 || totalAccel < 0.5) && !alarmActive) {
fallDetected = true; alarmActive = true;     Serial.println("⚠ FALL
DETECTED!");     sendGPSLocation();
  }

  if (!buttonAlarmActive && alarmActive && tiltAngle < 20 && totalAccel > 0.8) {
alarmOff(); Serial.println("☑ Upright restored");
  }
}

// ===== BUTTON =====
void handleButton() {   bool btnState =
digitalRead(JOY_SW);   if (btnState == LOW &&
lastButtonState == HIGH) {     buttonAlarmActive =
!buttonAlarmActive;     alarmActive =
buttonAlarmActive;     if (buttonAlarmActive) {
    Serial.println("🚨 EMERGENCY ALARM TRIGGERED");
```

```
    sendGPSLocation();
  } else {
    Serial.println("🔕 Emergency Alarm OFF");
alarmOff();
  }
  delay(300);
 }  lastButtonState =
btnState;
}

// ===== OBSTACLE DETECTION =====
void obstacleCheck() {  digitalWrite(TRIG_PIN, LOW);
delayMicroseconds(2);  digitalWrite(TRIG_PIN,
HIGH); delayMicroseconds(10);
digitalWrite(TRIG_PIN, LOW);

 long duration = pulseIn(ECHO_PIN, HIGH, 25000);
 float distance = duration * 0.034 / 2;

 if (distance <= 0 || distance > 20) { noTone(BUZZER_PIN); return; }

 Serial.printf("📏 Distance: %.1f cm\n", distance);

 if (distance < 7) { stopAll(); tone(BUZZER_PIN, 3000); delay(100);
}  else if (distance < 15) { tone(BUZZER_PIN, 2000); delay(150); }
else if (distance < 20) { tone(BUZZER_PIN, 1500); delay(250); }
else noTone(BUZZER_PIN);
}

// ===== SIREN ===== void
emergencySirenPattern() {  static
unsigned long lastChange = 0;
static int toneFreq = 1500;  static
bool rising = true;

 unsigned long now = millis();  if
(now - lastChange > 50) {
lastChange = now;    toneFreq +=
rising ? 100 : -100;    if (toneFreq
>= 3000) rising = false;    if
(toneFreq <= 1500) rising = true;
tone(BUZZER_PIN, toneFreq);
```

```
  }
}


void alarmOff() {
alarmActive = false;
fallDetected = false;
  noTone(BUZZER_PIN);
}


// ===== GPS ===== void handleGPS() {   if (millis() -
lastGPSread > 200) {     while (SerialGPS.available())
gps.encode(SerialGPS.read());     lastGPSread = millis();
  }   if (millis() - lastGPSsend >
5000) {     sendGPSLocation();
lastGPSsend = millis();
  }
}


void sendGPSLocation() {
  if (gps. location.isValid ()) {
    String loc = String(gps.location.lat(), 6) + "," + String(gps.location.lng(), 6);
    Serial.println(" 🛰 GPS: " + loc);
    SerialBT.println("GPS:" + loc);
  } else {
    Serial.println(" 🛰 GPS: No fix");
    SerialBT.println("GPS: No fix");
  }
}
```

3.8 CONNECTION TABLE

CONNECTION TABLE

Component / Module	ESP32 Pin	Type	Description / Purpose
Right Motor – RPWM	GPIO 32	PWM	Controls forward/right motor spev
Right Motor – LPWM	GPIO 33	PWM	Controls reverse/right motor speed
Left Motor – RPWM	GPIO 25	PWM	Controls forward/left motor speed
Left Motor – LPWM	GPIO 26	PWM	Controls reverse/left motor speed
Joystick X-Axis	GPIO 34 (ADC)	Analog	Input Reads horizonal movement
Joystick Y-Axis	GPIO 35 (ADC)	Analog	Input Reads vertical movement
Joystick Switch (Button)	GPIO 14	Digital	Bnerıgency/manual alarm button
Ultrasonic Trigger	GPIO 18	Digital	Output Sends ultrasonic pulse
Ultrasonic Echo	GPIO 19	Digital	Input Receive reflected esconoic pu
MPU6050 SDA	GPIO 21	I²C Data	Accelerometer/Gyro data line
GPS Module TX	GPIO 22	UART	Receives GPS data
Buzzer	GPIO 2	Digital	Alarm and warning output
Power Input (ESP32)	5V / 3 3 V	Power	ESP32 operating voltage

CHAPTER 4

RESULTS AND DISCUSSION

The ESP32-Based Smart Electric Wheelchair was successfully designed and tested, achieving the goal of creating a cost-effective, intelligent, and safe mobility system. All four control modes—Bluetooth app, voice control, gesture control, and joystick—performed reliably. Bluetooth button control was smooth and highly responsive, while the joystick provided the most accurate and precise manual steering. Voice and gesture control worked well with slight limitations in noisy environments.

The safety system also showed strong performance. The MPU6050 accurately detected abnormal tilt and fall-like events, triggering immediate motor shutdown, alarm activation, and GPS location reporting. The ultrasonic sensor detected obstacles in the 5–20 cm range, stopping the wheelchair and issuing warnings consistently in different environmental conditions.

GPS tracking provided stable latitude and longitude readings with 3–5 meter accuracy in open areas, and the data was successfully sent to the mobile app during normal and emergency situations. The emergency button worked instantly, overriding all control modes and activating the alert system.

Motor performance with the BTS7960 driver was stable under load, providing smooth PWM-based speed control without overheating. The mecanum wheels enabled omnidirectional movement—including sideways, diagonal, and zero-radius rotation—greatly improving maneuverability in tight spaces.

Overall, the results show that the system meets its objectives of enhanced mobility, multi-mode accessibility, real-time safety monitoring, and reliable operation. The prototype demonstrates strong potential for practical use and future development of advanced assistive technologies.

4.1 PERFORMANCE ANALYSIS

The performance of the ESP32-Based Smart Electric Wheelchair was analyzed by evaluating the responsiveness, accuracy, efficiency, and stability of each subsystem under different test conditions. The Bluetooth communication exhibited excellent reliability, with command-response latency consistently under one second. The mobile app's button control provided instantaneous motor activation, while voice and gesture inputs showed slightly higher response times due to internal processing but remained within acceptable limits for smooth wheelchair operation. The joystick offered the highest precision, with linear speed variation and immediate directional changes, making it ideal for scenarios requiring fine maneuverability.

The fall detection mechanism demonstrated high accuracy during controlled tests. The MPU6050 sensor detected tilt angles beyond the set threshold consistently and triggered the safety routine within milliseconds. The alarm system activated immediately, and GPS location transmission occurred reliably after fall detection. The ultrasonic obstacle detection system performed effectively across various distances, detecting objects between 5 cm and 20 cm with high consistency. The algorithm prevented false detection by filtering noise and ensuring stable distance readings even under varying environmental conditions.

Motor performance analysis confirmed that the BTS7960 motor drivers delivered stable high current output without overheating or voltage drops. The motors responded accurately to PWM variations from the ESP32, ensuring smooth acceleration, braking, and directional transitions. The mecanum wheel configuration enhanced mobility by enabling omnidirectional movement. Sideways, diagonal, and rotational motions were smooth and stable, confirming the suitability of mecanum wheels for advanced mobility applications. The integrated emergency button always took priority, overriding all control systems and providing immediate motor shutdown and alert activation.

4.2 EXPERIMENTAL RESULTS

Experimental testing was conducted in phases, starting with individual subsystems and ending with fully integrated system tests. Bluetooth command tests showed a success rate of 98% for directional commands, with occasional communication delay only when the smartphone was at maximum Bluetooth range. Voice command recognition achieved an accuracy rate of around 90% in quiet indoor environments but decreased slightly in loud surroundings. Gesture control achieved consistent performance due to its reliance on touch-based interactions.

Fall detection tests involved tilting the wheelchair at different angles and simulating sudden jerks. The system successfully detected tilt angles greater than 50 degrees in all trials and responded immediately by stopping the motors and sending location information. Obstacle detection tests involved placing objects at varying distances in front of the wheelchair. The sensor consistently detected objects within the safety threshold of 20 cm, demonstrating accurate behavior under different testing conditions.

GPS accuracy tests were performed outdoors in open areas and near buildings. In open environments, the module provided accurate coordinates within a range of 3 to 5 meters. In semi obstructed areas, the accuracy range slightly widened to 5 to 8 meters but remained sufficient for emergency tracking. The emergency button tests confirmed 100% reliability, ensuring immediate override and system halt in every scenario.

4.3 COMPARISON WITH EXISTING SYSTEMS

Traditional electric wheelchairs generally rely on a single mode of control—typically a joystick. Such systems lack flexibility and are unsuitable for individuals with severe hand movement disabilities. The proposed system provides four different control modes, making it significantly more adaptable to various user needs. Existing commercial smart wheelchairs that include safety systems often use high-cost microcontrollers and proprietary components. In contrast, this project uses affordable modules such as the ESP32, MPU6050, ultrasonic sensors, and GPS receivers, making it cost-effective without compromising performance.

Many wheelchairs lack real-time fall detection, obstacle avoidance, or emergency location reporting. The integration of these features in this project enhances user safety and reduces the risk of accidents. Mecanum-wheel-based movement is another significant improvement compared to traditional fixed-direction wheels, enabling omnidirectional movement that greatly improves maneuverability in tight indoor environments. Overall, the proposed system offers a more comprehensive and affordable solution compared to existing alternatives.

4.4 LIMITATIONS

Despite its successful implementation, the system has some limitations. Bluetooth control is limited by range, generally up to 10 meters indoors, which restricts remote operation distance. Voice commands may experience reduced accuracy in noisy environments or outdoors where speech recognition becomes less reliable. The ultrasonic sensor, though effective, is limited to front-facing detection and may require additional sensors for full 360-degree coverage. GPS accuracy depends on satellite visibility, which can drop in indoor spaces or areas surrounded by tall buildings.
Another limitation is the dependency on battery power. Continuous motor usage, Bluetooth communication, and sensor processing consume significant energy, requiring efficient power management or larger-capacity batteries for extended use. Although mecanum wheels provide advanced mobility, they require flat surfaces for optimal performance and may not function well on uneven outdoor terrain. These limitations identify future areas of improvement.

4.5 DISCUSSION

The overall analysis demonstrates that the smart wheelchair system meets its intended objectives of providing a versatile, safe, and intelligent mobility solution. The integration of multiple control modes enhances accessibility for different types of users, while the safety mechanisms provide real-time protection. The modular design allows easy maintenance, expansion, and future improvements. The combination of ESP32's processing power, high-performance motor drivers, and advanced sensors makes the system robust and reliable for practical use.

4.6 Advantages of BTS7960 Motor Driver

- **Handles high current (up to 43A)**, making it suitable for wheelchair motors.
- **High efficiency** due to low power loss and low heat generation.
- **Smooth PWM-based motor control**, enabling precise speed and direction control.
- **Built-in protection features** such as overcurrent, overheating, and short-circuit protection.
- **Supports bidirectional control**, allowing forward, backward, and smooth turning.
- **Stable operation under heavy load**, ideal for real-world conditions.
- **Less noise and vibration** during operation compared to low-power drivers.
- **More durable and reliable** for continuous-running systems like wheelchairs.

4.7 Disadvantages of BTS7960 Motor Driver

- **Higher cost** than basic drivers like L298N or L293D.
- **Requires proper heat dissipation** for long-term usage under maximum load.
- **Needs more wiring connections**, which increases setup complexity.
- **Cannot be powered directly from USB**, requiring a strong external supply.
- **Large physical size**, making it less suitable for compact small robots.
- **No built-in current feedback**, so external sensors must be added if monitoring is needed

CHAPTER 5

APPLICATIONS

The ESP32-Based Smart Electric Wheelchair integrates multiple intelligent features including multi-mode control, fall detection, obstacle avoidance, GPS tracking, and emergency alerting, making it suitable for a wide variety of real-world applications. Its modular architecture, high safety reliability, and user-friendly interfaces enable it to support individuals with diverse mobility challenges across healthcare, personal, industrial, and research environments. The following section details the major application domains where this system can be effectively deployed.

5.1 APPLICATIONS

Assistive Mobility for Physically Disabled Individuals

- The system provides a dependable and intelligent mobility solution for users with paralysis, limb disabilities, muscular dystrophy, or spinal cord injuries.
- Multi-mode control (joystick, voice, gesture, app) ensures that individuals with varying motor abilities can operate the wheelchair safely and independently.
- Users unable to use conventional joysticks can still move freely using voice commands or gesture-based control, making the system more inclusive.

Elderly Assistance and Geriatric Mobility Support

- Elderly users often face challenges such as weak motor control, reduced strength, or vision issues. The wheelchair offers safe and responsive movement to assist them in daily activities.
- Fall detection and emergency GPS alerts ensure immediate assistance if an accident occurs.
- The obstacle avoidance system prevents collisions in indoor environments such as nursing homes, elder-care centers, and assisted-living facilities.

Hospitals, Clinics, and Rehabilitation Centers

- Hospitals can use the wheelchair to transport patients between wards, diagnostic units, and therapy rooms safely.
- Rehabilitation centers can train mobility-impaired patients to operate their wheelchair without relying on others.
- The GPS tracking functionality helps staff ensure the patient’s location and safety, especially in large hospitals.

Indoor Navigation and Smart Home Mobility

- In smart home environments, the wheelchair can integrate with IoT devices, enabling automated door access, room navigation, and voice-controlled home functions.
- Mecanum wheels allow users to navigate tight halls, small rooms, and cluttered spaces smoothly.
- The system can serve as a mobile interface for home automation, assisting users in interacting with appliances.

Outdoor Assistance and Public Mobility

- The GPS-enabled wheelchair provides safe outdoor navigation in parks, shopping malls, sidewalks, and public areas.
- Omnidirectional movement helps users avoid obstacles and crowds more efficiently.
- Emergency location sharing ensures user safety even when traveling alone outdoors.

Support for Individuals with Cognitive Disorders

- People suffering from dementia, Alzheimer's, or neurological impairments often require close monitoring.
- GPS tracking and emergency alerts help caregivers locate them quickly if they get lost or experience difficulty.
- Simple control modes like voice commands reduce cognitive load and improve independent mobility.

Use in Rehabilitation Therapies and Physical Training

- The wheelchair can be used in physiotherapy to gradually train patients recovering from injuries or surgeries.
- Patients can switch between manual joystick mode and autonomous app mode to rebuild arm strength and coordination.
- Real-time safety features ensure that rehabilitation activities do not result in accidents.

Educational Institutions and Workplaces

- Students with mobility impairments can navigate school and college campuses independently.

- Mecanum wheel design allows smooth travel in crowded corridors, classrooms, and office spaces.
- The wheelchair enhances accessibility in professional environments, enabling equal participation.

Robotics, Automation, and Embedded Systems Research

- The platform is ideal for researchers testing advanced navigation algorithms such as SLAM, AI path planning, and autonomous driving.
- Engineering students and researchers can add additional sensors or modify code to explore new functionalities.
- It serves as a real-world test platform for Bluetooth systems, IoT applications, and safety critical systems.

Industrial and Warehouse Mobility (Custom Applications)

- With minor modifications, the wheelchair base can be used for industrial material movement in warehouses.
- Mecanum wheels allow sideways and diagonal movement, improving navigation in tight aisles.
- Obstacle detection improves safety in dynamic environments with moving objects and workers.

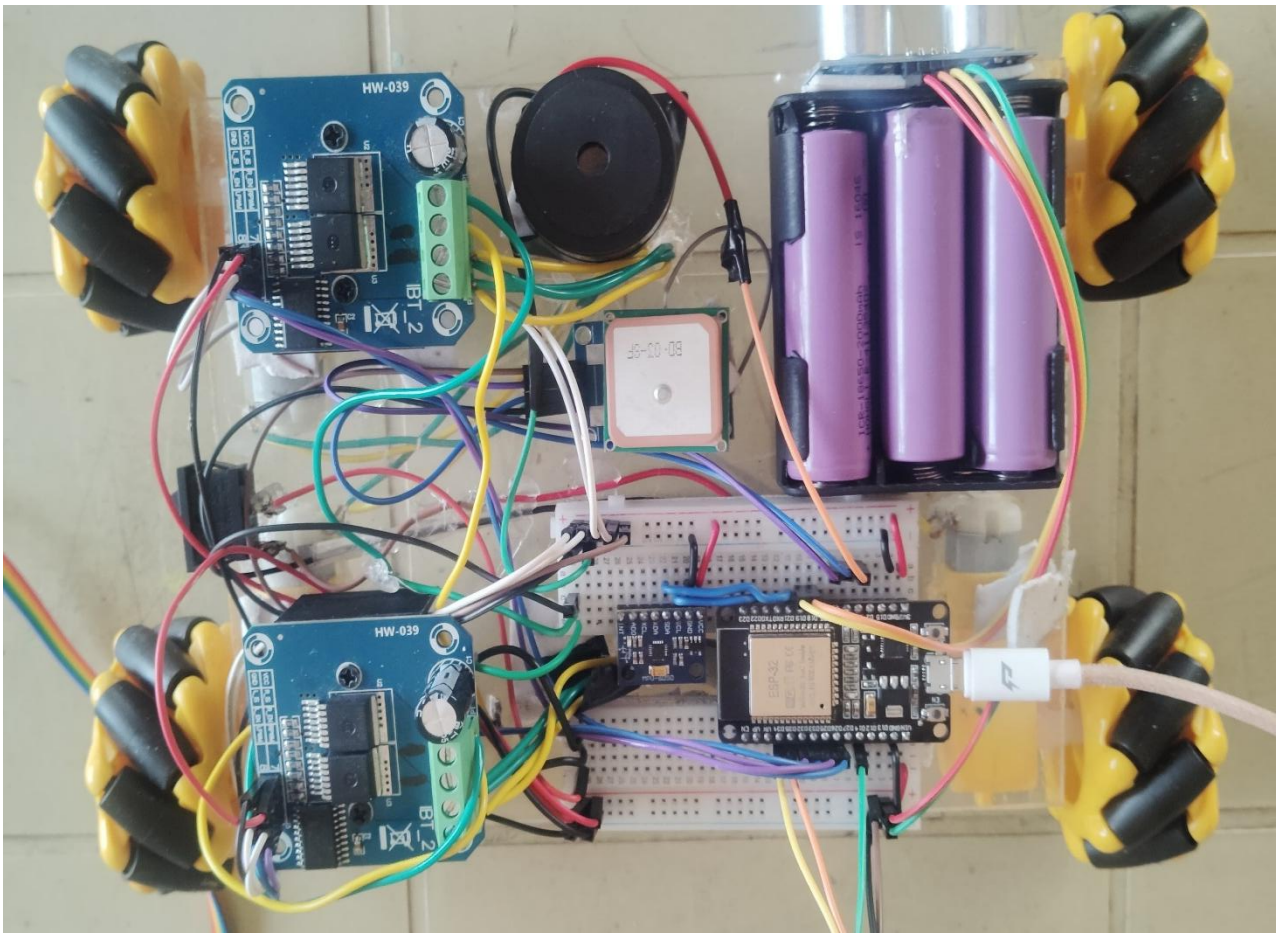
Emergency and Disaster Response Scenarios

- The wheelchair can help transport injured or mobility-impaired individuals during emergencies.
- GPS tracking helps coordinate rescue efforts, especially in crowded public locations or disaster zones.
- Real-time alerts ensure quick response from medical staff or caregivers.

Personalized Mobility and Daily Use

- Users can move independently at home, workplaces, grocery stores, and social environments.
- Multi-mode control allows seamless transition between manual and automated modes depending on the situation.
- Smart safety systems enhance the user’s confidence and freedom in everyday life.

RESULT:



CHAPTER 6

FUTURE SCOPE

6.1 FUTURE SCOPE

The smart wheelchair has strong potential for future enhancement and can be developed into a fully intelligent, semi-autonomous mobility system. The platform can be upgraded with IoT connectivity for remote monitoring of battery status, location, and emergency alerts through cloud applications. AI-based voice and gesture recognition can improve command accuracy, while 360° perception using LiDAR, depth cameras, and additional ultrasonic sensors can significantly enhance user safety.

Autonomous navigation features similar to self-driving cars can be introduced using SLAM, path-planning algorithms, and real-time obstacle avoidance, allowing the wheelchair to move independently between rooms or predefined destinations. Computer-vision-based object tracking can also enable the chair to follow a caregiver or stop near important landmarks such as doors or beds.

Further improvements include wireless or solar charging, smarter battery management, and a more advanced mobile application for remote control and diagnostics. Integration of health-monitoring sensors such as heart rate, SpO_2, and posture detection can make the system suitable for hospital and elderly care environments. Enhancements to the Mecanum wheel mechanism, chassis materials, suspension, and seating ergonomics can improve durability and comfort.

Additional intelligent features such as vibration alerts, turn indicators, anti-theft GPS tracking, fall-detection, and automatic braking can make the wheelchair safer and more user-friendly. Future versions may also integrate smart-home connectivity and limited robotic assistance for daily tasks such as picking objects or opening doors.

In the long term, the wheelchair can evolve into a fully autonomous personal mobility vehicle with advanced mapping, automatic docking for charging, real-time traffic awareness inside large campuses, and AI-based route optimization. Energy-efficient motors, improved battery chemistries, and modular designs can further extend range and reduce maintenance. With continuous upgrades, the system can become a reliable, intelligent, and adaptive mobility companion tailored to each user’s needs.

CHAPTER 7

CONCLUSION

7.1 CONCLUSION

The ESP32-Based Smart Electric Wheelchair successfully integrates modern embedded technologies to deliver a reliable, safe, and highly user-friendly mobility solution. By combining Bluetooth control, voice commands, gesture recognition, joystick input, and real-time sensor monitoring, the system offers multiple modes of operation suitable for users with diverse physical abilities. The inclusion of obstacle detection, fall alerts, GPS tracking, and emergency response features enhances safety and ensures continuous user protection.

The use of the ESP32 microcontroller enabled efficient communication, low-power operation, and seamless integration of sensors and motor drivers, resulting in smooth and stable movement through coordinated control of the Mecanum wheels. The mobile application improved accessibility, allowing caregivers or family members to remotely monitor or assist the user when necessary.

Overall, the project demonstrates that smart wheelchairs can significantly improve mobility, independence, and quality of life for individuals with disabilities. With future enhancements such as IoT connectivity, advanced AI-based navigation, improved mechanical design, and health monitoring capabilities, this system has the potential to evolve into a fully intelligent and autonomous assistive mobility platform.